PES-VCS: Version Control System

Author: Aanya K (PES2UG24CS011)

Sem: 4 Sec: A

System Architecture

This project implements a subset of Git's core functionality in C. It manages data through a Content-Addressable Storage (CAS) system, using SHA-1 hashing to ensure data integrity and deduplication.

Core Components:

- **Object Store:** Handles sharded storage of Blobs, Trees, and Commits in .pes/objects.
- **Index (Staging Area):** A persistent cache that tracks changes between the working directory and the next commit.
- **Tree Logic:** Recursively captures directory states and serializes them into binary objects.
- **Commit Logic:** Creates snapshots with parent-child relationships to maintain a verifiable history log.

Phase 1: Object Store Implementation

Goal: Implement sharding to prevent directory performance degradation. * **Logic:** Uses the first 2 characters of a hash as a sub-directory name and the remaining 38 as the filename.

* **Screenshot 1A:** Validation of core object storage and retrieval.

```
[04/16/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
[04/16/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed02
21bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea
09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: Evidence of the .pes/objects directory sharding.

```
[04/16/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ find .pes/objects -
type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5
e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1a
f8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650e
da471e566fcf
[04/16/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ git add object.c
```

Phase 2: Tree Objects & Serialization

Goal: Capture the state of a directory at a specific point in time. * **Logic:** Trees store entries (mode, type, hash, name). They are stored as binary objects to maintain compact metadata.

* **Screenshot 2A:** Recursive tree construction tests passing.

```
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$
```

* **Screenshot 2B:** Binary inspection of a tree object using xxd.

```
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/23/993c9206cde6476613eb2c35cafc23b955c3d96dd14b7b55a66734fcebb2c6
.pes/objects/26/f76b687d19f5dfedffef6ff1afb895f4295ae6245e29a167b5e33585247846
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d5/522b063647a1bb7bc1a944fae7e8e142f7c7169c64afe3c4f0983c712ecc7b
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ xxd .pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442 | head -n 2
0
00000000: 7472 6565 2034 3900 3130 3036 3434 2068  tree 49.100644 h
00000010: 656c 6c6f 2e74 7874 0066 2246 63d2 3e6f  ello.txt.f"Fc.>o
00000020: 4d9d e9e2 c7e6 d876 4305 a92a 3830 a1a5  M......vC..*80..
00000030: 2d3d 5f4a a800 7b5c 39                   -=_J..{\9
```

Phase 3: Staging Area (The Index)

Goal: Manage a "middle ground" before committing. * **Logic:** The index is a text-based manifest of filenames and their corresponding hashes, allowing for partial commits.

* **Screenshot 3A:** Execution of init, add, and status.

```
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ rm -rf .pes
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ echo "hello" > file1.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ echo "world" > file2.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes add file1.txt file2.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  Makefile
  untracked:  index.h
  untracked:  tree.c
  untracked:  test_tree
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  commit.c
  untracked:  index.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  commit.h
  untracked:  pes.h
```

* **Screenshot 3B:** Internal view of the index file.

```
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776403433 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776403444 6 file2.txt
```

Phase 4: Commits, Logs, and Refs

Goal: Linking snapshots to create a navigable history. * **Logic:** Commits contain a root tree hash, a parent hash (for history), and metadata.

* **Screenshot 4A:** pes log showing the chain of three sequential commits.

```
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ echo "Hello" > hello.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes add hello.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes commit -m "Initial commit"
Committed: d5522b063647... Initial commit
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ echo "World" >> hello.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes add hello.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes commit -m "Add world"
Committed: 26f76b687d19... Add world
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ echo "Goodbye" > bye.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes add bye.txt
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 23993c9206cd... Add farewell
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ ./pes log
commit 23993c9206cde6476613eb2c35cafc23b955c3d96dd14b7b55a66734fcebb2c6
Author: Aanya K <PES2UG24CS011>
Date:   1776404502

    Add farewell

commit 26f76b687d19f5dfedffef6ff1afb895f4295ae6245e29a167b5e33585247846
Author: Aanya K <PES2UG24CS011>
Date:   1776404495

    Add worldell

commit d5522b063647a1bb7bc1a944fae7e8e142f7c7169c64afe3c4f0983c712ecc7b
Author: Aanya K <PES2UG24CS011>
Date:   1776404484

    Initial commit
```

* **Screenshot 4B:** Verification of object growth in the storage layer.

```
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/23/993c9206cde6476613eb2c35cafc23b955c3d96dd14b7b55a66734fcebb2c6
.pes/objects/26/f76b687d19f5dfedffef6ff1afb895f4295ae6245e29a167b5e33585247846
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d5/522b063647a1bb7bc1a944fae7e8e142f7c7169c64afe3c4f0983c712ecc7b
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

* **Screenshot 4C:** Inspection of HEAD and branch reference files.

```
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ cat .pes/refs/heads/main
23993c9206cde6476613eb2c35cafc23b955c3d96dd14b7b55a66734fcebb2c6
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final Integration

- **Full Integration Test:** Evidence of the complete automated test suite passing.

```
[04/17/26]seed@VM:~/.../PES2UG24CS011-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 5448967cd485... Initial commit

Log after first commit:
commit 5448967cd4850451df8b99b98e73fb6aebbe772494147dead8adfe63b9cf58b2
Author: Aanya K <PES2UG24CS011>
Date:   1776404525

    Initial commit
```

-

```
--- Second Commit ---
Committed: 4e712e064dc6... Update file.txt

--- Third Commit ---
Committed: 9b47656498af... Add farewell

--- Full History ---
commit 9b47656498afa83fc08cf0aa7577209849d5fd1d1594da6e912dc7c7af5bf019
Author: Aanya K <PES2UG24CS011>
Date:   1776404525

    Add farewell

commit 4e712e064dc6d2e68de23bd688c742457fcfe284f5df58730c8a584b4220035b
Author: Aanya K <PES2UG24CS011>
Date:   1776404525

    Update file.txt

commit 5448967cd4850451df8b99b98e73fb6aebbe772494147dead8adfe63b9cf58b2
Author: Aanya K <PES2UG24CS011>
Date:   1776404525

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
9b47656498afa83fc08cf0aa7577209849d5fd1d1594da6e912dc7c7af5bf019

--- Object Store ---
Objects created:
10
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/4e/712e064dc6d2e68de23bd688c742457fcfe284f5df58730c8a584b4220035b
.pes/objects/54/48967cd4850451df8b99b98e73fb6aebbe772494147dead8adfe63b9cf58b2
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/9b/47656498afa83fc08cf0aa7577209849d5fd1d1594da6e912dc7c7af5bf019
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Technical Analysis

Section 5: Branching & Checkout Dynamics

Q5.1: How is pes checkout implemented and why is it complex? The checkout command transitions the workspace between different commit states. It involves: 1. Updating .pes/HEAD to point to the new branch ref. 2. Reading the commit hash from the target branch. 3. Recursively traversing the associ- ated Tree object. 4. Overwriting files in the working directory with the Blob data from the object store. **Complexity:** The primary challenge is "safety." We must ensure that uncommitted changes (dirty files) are not accidentally over- written and that files present in the current branch but missing in the target branch are correctly deleted from the disk.

Q5.2: Detecting Dirty Working Directory Conflicts Before a checkout or merge, the VCS must check for "dirty" states. This is done by comparing the file's hash in the Index against a fresh hash of the file currently on Disk. If these hashes differ, the file is modified. If that modified file also differs from the version in the Target Branch, the system must abort to prevent data loss.

Q5.3: Detached HEAD State A "Detached HEAD" occurs when the .pes/HEAD file contains a raw SHA-1 commit hash instead of a symbolic reference like ref: refs/heads/main. This happens when checking out a specific commit. While you can still commit in this state, those commits have no branch pointing to them and may be lost if you switch away. Recovery requires finding the hash in the log and creating a new branch manually: pes branch .

Section 6: Garbage Collection (GC) Strategy

Q6.1: Mark-and-Sweep Algorithm Design For a large repository (100k commits), I would implement a Mark-and-Sweep GC: 1. **Mark Phase:** Start at all "known roots" (files in .pes/refs/heads/ and .pes/HEAD). Recursively follow pointers from Commits to Trees, and Trees to Blobs. Tag every reached hash. 2. **Sweep Phase:** Iterate through the physical files in .pes/objects. Any file not tagged in the Mark phase is "unreachable" (dangling) and is safely deleted. This prevents the object store from bloating with obsolete data from deleted branches or failed operations.

Q6.2: Handling Race Conditions during GC A race condition occurs if a user creates a new object (e.g., during a large add operation) at the exact moment the GC Sweep begins. If the Commit object referencing those new Blobs hasn't been written yet, the GC might incorrectly identify the new Blobs as unreachable. **Prevention:** Git (and PES-VCS) implements a "Grace Period" (typically 2 weeks or a specific timestamp check). The Sweep phase will only delete unreachable objects that are older than a certain threshold, ensuring that "in-flight" objects are preserved.